

UNIVERSITÀ DI PISA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Master's Degree in Artificial Intelligence and Data Engineering

CLOUD COMPUTING

Amazon Reviews

Ginevri Erica
Nasoni Chiara
Vellutini Vittoria

1. Application Scenario **3**
2. Dataset Description **3**
3. Architecture Description **3**
4.1 Job 1: Product Aggregation 4
4.2 Job 2: Product Scoring and Ranking 4
4.3 Job 3: Anomaly Detection 5
5. REST Architecture Description **6**
5.1 REST client interface 6
5.2 Containerized REST server 6
5.3 Distributed Backend 7
6. Experimental Evaluation **7**
6.1 Hadoop MapReduce Performance Analysis 7
6.2 Apache Spark Performance Analysis 8
6.3 Performance of Python-based non-parallel execution 8
7. Discussion of Results **8**

1. Application Scenario

The project analyzes large-scale Amazon product reviews using distributed data processing.
The goal is to identify top-ranked products and detect anomalous products through a multi-stage analytics pipeline implemented with Hadoop MapReduce and Apache Spark.

The system processes Amazon review data and computes:

- aggregated product statistics;
- category-based ranking;
- anomaly detection on suspicious products.

2. Dataset Description

The dataset used is the Amazon US Customer Reviews Dataset available on Kaggle.
To ensure data quality and statistical significance for our workflow, we performed an initial data cleaning phase:

- **Category Selection:** we choose five specific categories: *Beauty, Music, Toys, Sports*, and *Shoes*.
- **Attributes Removal:** we removed unnecessary fields such as marketplace, vine, review_headline, and review_body.
- **Noise Reduction:** we removed products with fewer than 50 total reviews. Retaining products with only a handful of reviews would skew the ranking system (e.g., a product with a single 5-star review would obtain a perfect score, bypassing heavily reviewed products).

After applying these filters, the final dataset size is **1.29 GB**. The remaining fields for analysis are: customer_id, review_id, product_parent (acting as the unique product ID), product_title, product_category, star_rating, helpful_votes, total_votes, verified_purchase, and review_date.

3. Architecture Description

The system is based on a distributed Hadoop cluster configured with one master node (NameNode) and two worker nodes (DataNodes). HDFS is used for distributed storage while YARN manages cluster resources and job execution.

The analytics workflow is composed of three sequential jobs:

1. Product aggregation;
2. Product scoring and ranking;
3. Anomaly detection.

The same workflow was implemented both in:

- Hadoop MapReduce (Java);
- Apache Spark (Python).

A local Python implementation was also developed to compare distributed and non-distributed execution performance.

4. MapReduce Workflow Pseudocode

4.1 Job 1: Product Aggregation

Objective: read the raw TSV dataset and compute aggregated product statistics.

Optimization:

1. A **Combiner** is utilized to perform local partial aggregations on the Mapper nodes. This reduces the volume of data shuffled across the network.
2. A **Custom Writable** (*ReviewStatsWritable*) is used to efficiently transfer statistics between tasks.
3. **Multiple Reducers** distribute different *product_parent* keys across multiple reducers.

Execution and Pseudocode:

1. **Mapper**: Reads the TSV dataset, extracts the relevant review fields, and emits: (*product_parent*, *ReviewStatsWritable*)
2. **Combiner:** Locally aggregates partial statistics for the same product on each Mapper.
3. **Reducer:** performs the final aggregation, computes the mathematical average and emits (*product_parent*, *aggregatedMetrics*). The formula used is: $avg_rating = \frac{rating_sum}{review_count}$

The final aggregated statistics are saved to HDFS and used as input for Job 2 and Job 3.

Algorithm 1 Job 1: Product Aggregation (Map–Combine–Reduce)

Require: Input TSV dataset, output path
Ensure: Aggregated statistics for each product

```
1:  class Mapper
2:      method MAP(offset, line)
3:          if line is invalid then return
4:          product_parent, reviewStats ← extractMetrics(line)
5:          EMIT(key=product_parent, value=reviewStats)

6:  class Combiner                                  ▷ Local partial aggregation
7:      method REDUCE(product_parent, stats_list)
8:          local_stats ← emptyStats()
9:          for each stats in stats_list do
10:             local_stats ← local_stats + stats
11:         EMIT(key=product_parent, value=local_stats)

12: class Reducer                                   ▷ Global final aggregation
13:     method REDUCE(product_parent, stats_list)
14:         final_stats ← emptyStats()
15:         for each stats in stats_list do
16:             final_stats ← final_stats + stats
17:         avg_rating ← final_stats.ratingSum / final_stats.reviewCount
18:         result ← formatOutput(final_stats, avg_rating)
19:         EMIT(key=product_parent, value=result)
```

4.2 Job 2: Product Scoring and Ranking

Objective: calculate a weighted score for each product and rank them within their category.

Optimization:

1. A **Custom Partitioner** guarantees that all products of the same category are processed by the same Reducer, allowing a correct category-level ranking.
2. A **Custom WritableComparable** (*CategoryScoreKey*) is used to sort the products by category and score descending before they reach the Reducer.
3. **Multiple Reducers** allow different categories to be ranked in parallel across the cluster.

Execution and Pseudocode:

- **Mapper:** reads the aggregated statistics produced by Job 1, computes a product score, and emits (*CategoryScoreKey*(category, score), product information).
  The score formula used is:
  $score = avg_rating \cdot log(1 + review_count) \cdot (1 + \frac{helpful_votes}{1+ total_votes})$
- **Partitioner:** partitions records using the product category.
- **Reducer:** Each reducer receives products already sorted by category and descending score, assigns a ranking position within each category, and emits (*product_parent*, *product_title*, *category*, *score*, *rank*)

The ranked products are stored in HDFS and used as input for Job 3.

Algorithm 2 Job 2: Product Ranking

Require: Aggregated statistics from Job 1, output path
Ensure: Ranked products within each category

```
1: class Mapper
2:   method MAP(offset, line)
3:     if line is invalid then return
4:     category, score, productInfo ← computeScore(line)
5:     EMIT(key=(category, score), value=productInfo)

6: class Partitioner                          ▷ Partition by category
7:   method GETPARTITION(category, score)
8:     return hash(category)

9: class Reducer                                    ▷ Assign ranking
10:  method REDUCE(category, rankedProducts)
11:    rank ← 1
12:    for each product in rankedProducts do
13:      EMIT(product_parent, product, score, rank)
14:      rank ← rank + 1
```

4.3 Job 3: Anomaly Detection

Objective: identify products with suspicious review patterns or ranking inconsistencies.

Optimization:

1. The **setup()** method loads thresholds from the Hadoop configuration.
2. The **cleanup()** method generates a summary of the anomalies detected by each Reducer.
3. **Multiple Reducers** distribute different *product_parent* keys across multiple reducers.

Execution and Pseudocode:

- **Mapper:** reads the outputs of Job 1 and Job 2, tags each record according to its source, and emits (*product_parent*, *tagged record (JOB1 or JOB2)*)
- **Reducer:** computes anomaly metrics, and applies four detection rules:
  - *Helpful Ratio Anomaly*: high score and low ratio of helpful votes over total votes.
  - *Verified Ratio Anomaly*: many reviews but a low verified purchases.
  - *Suspicious Perfect Product*: almost perfect avg_rating and large number of reviews.
  - *Rank Inconsistency*: products with high ranking and relatively low average rating.

  Emits (*product_parent*, *anomaly*, *product_title*, *category*, *score*).

Detected anomalies are written to HDFS together with the anomaly type and product information.

Algorithm 3 Job 3: Anomaly Detection

Require: Job 1 output, Job 2 output, output path
Ensure: Detected anomalous products

```
1: class Mapper
2:   method MAP(offset, line)
3:     if line belongs to Job1 then
4:       EMIT(product_parent, JOB1_record)
5:     else if line belongs to Job2 then
6:       EMIT(product_parent, JOB2_record)

7: class Reducer                                   ▷ Anomaly Detection
8:   method REDUCE(product_parent, records)
9:     productData ← empty
10:    rankingData ← empty
11:    for each record in records do
12:      if record comes from Job1 then
13:        productData ← extractMetrics(record)
14:      else
15:        rankingData ← extractRanking(record)
16:    helpfulRatio ← computeHelpfulRatio()
17:    verifiedRatio ← computeVerifiedRatio()
18:    if anomalyDetected() then
19:      EMIT(product_parent, anomalyInfo)
```

5. REST Architecture Description

The distributed analytics functionalities are exposed through a REST-oriented architecture composed of three main components: a REST client interface, a containerized REST server and a distributed backend based on Hadoop and Spark.

5.1 REST client interface

The client interface is a web-based dashboard (implemented as a single-page HTML/JavaScript application) that allows users to interact with the system through a browser.

The interface enables users to select the input dataset, configure execution parameters for each single job like the number of reducers, the input split size, the anomaly thresholds, launch Hadoop pipelines and Spark jobs, monitor execution progress in real time via progress bars, and retrieve results and execution statistics through a job history table with a detail modal.

5.2 Containerized REST server

The Containerized REST server is developed using Flask. The server acts as an intermediate layer between the client and the distributed backend. Its main responsibilities are:

- exposing REST endpoints;
- orchestrating Hadoop and Spark job execution;
- monitoring job status;
- collecting and returning execution results.

The REST server runs inside a Docker container, exposing port 8080 to allow the client interface to access the APIs. To execute jobs, the server connects to the cluster via SSH.

The server exposes the following endpoints:

Method	Endpoint	Description
POST	`/jobs/hadoop`	Launches the Hadoop pipeline (Job1 → Job2 → Job3)
GET	`/jobs/hadoop/pipeline/<pipeline_id>`	Monitors the progress of a pipeline
GET	`/jobs/hadoop`	Lists Hadoop jobs
GET	`/jobs/hadoop/<job_id>/result`	Retrieves the output of a Hadoop job
POST	`/jobs/spark`	Launches the Spark job
GET	`/jobs/spark`	Lists Spark jobs
GET	`/jobs/spark/<job_id>/result/<output_type>`	Retrieves the output of a Spark job (ranking or anomalies)
GET	`/jobs/<job_id>`	Returns the details of a job (Hadoop or Spark)
GET	`/`	Returns the HTML page

The REST API endpoints are not directly invoked by the end user. The user interacts with the web interface (frontend), which acts as a client of the REST server.

5.3 Distributed Backend

The Hadoop cluster (one NameNode and two DataNodes) and the Spark runtime run on three virtual machines. The REST server interacts with them exclusively through SSH for job submission and through their native REST APIs (YARN on port 8088, WebHDFS on port 9870) for monitoring and result collection.

6. Experimental Evaluation

The performance of the multi-stage analytics pipeline was evaluated using two dataset scales: a sampled subset of 250 MB and the complete dataset of 1.29 GB (Full Dataset).

6.1 Hadoop MapReduce Performance Analysis

The Hadoop implementation was tested by tuning the Input Split Size and the number of Reducers.

Dataset	Split Size	Reducers	Computational Step	Execution Time (ms)	Memory-Seconds (MB-s)	vCore-Seconds
250 MB	128 MB	2	Job 1 - Aggregation	24,398	23,804	59
			Job 2 - Ranking	21,805	19,958	47
			Job 3 - Anomaly Detection	23,507	31,355	89
250 MB	128 MB	4	Job 1 - Aggregation	27,936	30,466	80
			Job 2 - Ranking	35,508	43,809	123
			Job 3 - Anomaly Detection	49,735	115,996	388
250 MB	64 MB	2	Job 1 - Aggregation	60,824	72,380	212
			Job 2 - Ranking	24,732	22,951	56
			Job 3 - Anomaly Detection	27,756	36,285	106
250 MB	64 MB	4	Job 1 - Aggregation	27,721	36,241	102
			Job 2 - Ranking	35,977	47,206	137
			Job 3 - Anomaly Detection	28,548	56,909	181
Full Dataset	128 MB	2	Job 1 - Aggregation	75,294	194,807	669
			Job 2 - Ranking	25,509	23,753	59
			Job 3 - Anomaly Detection	39,414	48,969	144
Full Dataset	128 MB	4	Job 1 - Aggregation	52,977	175,380	460
			Job 2 - Ranking	32,812	47,705	141
			Job 3 - Anomaly Detection	32,441	68,672	221
Full Dataset	64 MB	2	Job 1 - Aggregation	63,394	232,977	831
			Job 2 - Ranking	25,160	21,738	51
			Job 3 - Anomaly Detection	24,044	27,784	75
Full Dataset	64 MB	4	Job 1 - Aggregation	96,175	316,663	1,119
			Job 2 - Ranking	30,086	39,057	110
			Job 3 - Anomaly Detection	33,524	66,850	212

On the 250 MB dataset, doubling the reducers from 2 to 4 actually hurt performance. Because the data volume is so small, the cluster spent more time setting up and allocating containers than doing actual work.

For the full dataset, in the aggregation phase, increasing the number of reducers improved execution time by distributing the larger workload across more tasks. However, the ranking and anomaly detection stages did not benefit to the same extent, as they operate on already aggregated data and therefore have less work to parallelize.

The experiments also show that reducing the input split size from 128 MB to 64 MB did not consistently improve performance. While smaller splits increase the level of parallelism, they also generate a larger number of map tasks, leading to higher scheduling costs and greater memory and CPU usage.

6.2 Apache Spark Performance Analysis

Dataset	Execution Time (ms)	Memory-Seconds (MB-s)	vCore-Seconds
250 MB	50,864	114,550	126
Full Dataset	80,065	183,847	204

The results show that Spark handles the larger dataset efficiently. Although the full dataset required more execution time and consumed more resources than the 250 MB dataset, the increase was significantly smaller than the increase in data volume. This behavior highlights Spark's ability to scale effectively through distributed in-memory processing, reducing the impact of disk I/O and minimizing the overhead associated with iterative computations.

6.3 Performance of Python-based non-parallel execution

Dataset	Execution Time (Seconds)
250 MB	11.94 s
Full Dataset	60.33 s

The Python implementation achieved the lowest execution time on both dataset sizes. Since it runs locally on a single machine, it avoids the overhead associated with distributed execution, such as resource allocation and task scheduling. For the considered workload, these overheads outweigh the benefits of parallel processing, especially on the smaller dataset.

7. Discussion of Results

The comparison between Hadoop and Spark shows that Spark consistently outperformed Hadoop when considering the execution time of the entire analytics workflow. For both dataset sizes, Spark completed the ranking and anomaly detection pipeline faster than Hadoop, despite executing the same analytical tasks. This result can be attributed to Spark's in-memory processing model, which avoids repeatedly writing intermediate results to HDFS between computational stages.

These results confirm one of the main advantages of Spark over Hadoop MapReduce for multi-stage analytics workflows: reducing disk I/O and job orchestration overhead leads to significantly shorter end-to-end execution times, especially when multiple processing stages are involved.